



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION



Date: 22/09/26

Student Name: MEHAK

Roll No.: 2501410004

Course/Semester: BTECH CSE CYBER SECURITY SEM-3

PROJECT 1 :

1. LAB1:

a. NOT gate :

CHIP Not {

IN in;

OUT out;

PARTS:

Nand(a=in , b=in , out=out);

}

NAND2Tetris / Hardware Simulator / Not.hdl

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/1/Not.hdl
5 /**
6  * Not gate:
7  * if (in) out = 0, else out = 1
8  */
9 CHIP Not {
10     //names (no change)
11     IN in;
12     OUT out;
13     //actual implementation
14     PARTS:
15         Nand(a=in , b=in , out=out);
16     //// Replace this comment with your code
```

Chip Not

Input pins

in 1

Output pins

out 0

Test Script

Compare File

Output File

Diff Table

```
6 output <= in out;
9
10 set in 0;
11 eval;
12 output;
13
14 set in 1;
15 eval;
16 output;
```

Simulation successful: The output file is identical to the compare file

b. AND gate:

```
CHIP And {
```

```
    IN a, b;
```

```
    OUT out;
```

```
    PARTS:
```

```
        Nand(a=a, b=b, out=nandOut);
```

```
        Nand(a=nandOut, b=nandOut, out=out);
```

```
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top browser window shows the URL `nand2tetris.github.io/web-ide/chip/`. The simulator's title bar reads "NAND2Tetris / Hardware Simulator / And.hdl". The main editor area contains the HDL code for the `And` chip, which implements an AND gate using two NAND gates. The code includes comments and a `PARTS` section. To the right of the code editor is a control panel for the `Chip And`, featuring buttons for "Eval", "Reset", and "Clock: 0". Below these are sections for "Input pins" (a and b, both set to 1), "Output pins" (out, set to 1), and "Internal pins" (nandOut, set to 0). At the bottom, a "Test" panel shows a test script with steps 20 through 28, including `set`, `eval`, and `output` commands. A status bar at the very bottom indicates "Simulation successful: The output file is identical to the compare file".

c. Or gate:

```
CHIP Or {  
    IN a, b;  
    OUT out;
```

PARTS:

```
Nand(a=a, b=a, out=notA);  
Nand(a=b, b=b, out=notB);  
Nand(a=notA, b=notB, out=out);  
}
```

The screenshot shows the NAND2Tetris web IDE interface. The top browser tabs include 'NAND2TETRIS - Google Docs', 'NAND2Tetris', and 'KRMU LOGO - Google Search'. The address bar shows 'nand2tetris.github.io/web-ide/chip/'. The main interface has a title bar 'NAND2Tetris / Hardware Simulator / Or.hdl'. Below this is a code editor with the following HDL code:

```
HDL --LBSt Project 1 Or  
1 // This file is part of www.nand2tetris.org  
2 // and the book "The Elements of Computing Systems"  
3 // by Nisan and Schocken, MIT Press.  
4 // File name: projects/1/Or.hdl  
5 /**  
6  * Or gate:  
7  * if (a or b) out = 1, else out = 0  
8  */  
9 CHIP Or {  
10     IN a, b;  
11     OUT out;  
12  
13     PARTS:  
14     Nand(a=a, b=a, out=notA);  
15     Nand(a=b, b=b, out=notB);  
16     Nand(a=notA, b=notB, out=out);  
17 }
```

To the right of the code editor is a 'Chip Or' panel with the following pins:

Input pins	
a	1
b	1
Output pins	
out	1
Internal pins	
notA	0
notB	0

Below the code editor is a 'Test' panel with a 'Run' button and a 'Steps: 7 Outputs: 5' indicator. The test script shows the following commands:

```
20 set a 1,  
21 set b 0,  
22 eval,  
23 output;  
24  
25 set a 1,  
26 set b 1,  
27 eval,  
28 output;
```

At the bottom, a green status bar indicates: 'Simulation successful: The output file is identical to the compare file'.

2. LAB2:

a. XOR gate:

```
CHIP Xor {  
    IN a, b;  
    OUT out;
```

PARTS:

```
Not(in=a, out=nota);
```

```
Not(in=b, out=notb);
```

```
And(a=a, b=notb, out=C);
```

```
And(a=nota, b=b, out=D);
```

```
Or(a=C, b=D, out=out);
```

```
}
```

The screenshot shows the NAND2Tetris Hardware Simulator interface. The top browser window displays the URL `nand2tetris.github.io/web-ide/chip/`. The simulator title is "NAND2Tetris / Hardware Simulator / Xor.hdl". The main editor shows the HDL code for the Xor chip, which is a copy of the code provided in the previous blocks. The right sidebar shows the chip's pins: Input pins (a=1, b=1), Output pins (out=0), and Internal pins (nota=0, notb=0, C=0, D=0). The bottom status bar indicates "Steps: 7 Outputs: 5 Slow" and "Fast". The Test Script tab shows a sequence of commands: `set a 1, set b 0, eval, output;` followed by `set a 1, set b 1, eval, output;`. A green message at the bottom states: "Simulation successful: The output file is identical to the compare file".

b. MUX gate:

```
CHIP Mux {  
  
    IN a, b, sel;  
  
    OUT out;  
  
    PARTS:  
    Not(in=sel, out=C);  
    And(a=a, b=C, out=x);  
    And(a=b, b=sel, out=y);  
    Or(a=x, b=y, out=out);  
}
```

The screenshot shows the NAND2Tetris Hardware Simulator interface. The top browser window displays the URL `nand2tetris.github.io/web-ide/chip/`. The simulator window has a title bar `NAND2Tetris / Hardware Simulator / Mux.hdl`. The main editor shows the HDL code for the Mux chip, which is identical to the code provided in the previous block. The right sidebar shows the chip's configuration: **Chip Mux** with buttons for `Eval`, `Reset`, and `Clock: 0`. Below this, the input pins are `a` (1), `b` (1), and `sel` (1). The output pins are `out` (1). The internal pins are `C` (0), `x` (0), and `y` (1). The bottom section shows the test script with the following code:

```
40 set a 1,  
41 set b 1,  
42 set sel 0,  
43 eval,  
44 output;  
45  
46 set sel 1,  
47 eval,  
48 output;
```

Below the test script, a green status bar indicates: `Simulation successful: The output file is identical to the compare file`.

c. DMux gate:

```
CHIP DMux {  
    IN in, sel;  
    OUT a, b;
```

PARTS:

```
Not(in=sel, out=nsel);  
And(a=in, b=nsel, out=a);  
And(a=in, b=sel, out=b);  
}
```

The screenshot shows the NAND2Tetris Hardware Simulator interface. The top browser window displays the URL `nand2tetris.github.io/web-ide/chip/`. The simulator's title bar reads "NAND2Tetris / Hardware Simulator / DMux.hdl". The main editor shows the HDL code for the DMux chip, which includes comments about its source and a Verilog-style implementation using a demultiplexer and logic gates. The right-hand panel, titled "Chip DMux", shows the pin configuration: input pins `in` (1) and `sel` (1), output pins `a` (0) and `b` (1), and an internal pin `nsel` (0). The bottom section, labeled "Test", shows a test script with steps 19 through 26, including setting inputs, evaluating, and outputting. A green status bar at the bottom indicates "Simulation successful: The output file is identical to the compare file".

```
1 // This file is part of www.nand2tetris.org  
2 // and the book "The Elements of Computing Systems"  
3 // by Nisan and Schocken, MIT Press.  
4 // File name: projects/1/DMux.hdl  
5 /**  
6  * Demultiplexor:  
7  * [a, b] = [in, 0] if sel = 0  
8  *           [0, in] if sel = 1  
9  */  
10 CHIP DMux {  
11     IN in, sel;  
12     OUT a, b;  
13  
14     PARTS:  
15     Not(in=sel, out=nsel);  
16     And(a=in, b=nsel, out=a);  
17     And(a=in, b=sel, out=b);  
18 }
```

Test Script: Steps: 7 Outputs: 5 Slow Fast

```
19 set in 1,  
20 set sel 0,  
21 eval,  
22 output;  
23  
24 set sel 1,  
25 eval,  
26 output;
```

Simulation successful: The output file is identical to the compare file

3. LAB3:

a. Not16:

```
CHIP Not16 {  
  
    IN in[16];  
  
    OUT out[16];  
  
PARTS:  
  
    Nand16(a=in, b=in, out=out);  
  
    ///// Replace this comment with your code.  
  
}
```

The screenshot shows the NAND2Tetris Hardware Simulator interface. The top browser window displays the URL `nand2tetris.github.io/web-ide/chip/`. The main interface has a title bar `NAND2Tetris / Hardware Simulator / Not16.hdl`. The left pane shows the HDL code for `Not16.hdl`, which includes comments about the source and a placeholder for the implementation. The right pane shows the `Chip Not16` configuration with input pins `in` (0001001000110100) and output pins `out` (1110110111001011). The bottom pane shows the test script with steps 8 and 6, and a status bar indicating a successful simulation.

HDL Code:

```
1 // This file is part of www.nand2tetris.org  
2 // and the book "The Elements of Computing Systems"  
3 // by Nisan and Schocken, MIT Press.  
4 // File name: projects/01/Not16.hdl  
5 /**  
6  * 16-bit Not gate:$D  
7  * for i = 0, ..., 15:  
8  * out[i] = Not(a[i])  
9  */  
10 CHIP Not16 {  
11     IN in[16];  
12     OUT out[16];  
13  
14     PARTS:  
15     Nand16(a=in, b=in, out=out);  
16     ///// Replace this comment with your code
```

Chip Not16 Configuration:

- Input pins: in = 0001001000110100 (dec)
- Output pins: out = 1110110111001011 (dec)

Test Script:

```
21  
22 set in %B0011110011000011,  
23 eval,  
24 output;  
25  
26 set in %B0001001000110100,  
27 eval,  
28 output;
```

Status: Simulation successful: The output file is identical to the compare file

b. And16:

```
CHIP And16 {  
    IN a[16], b[16];  
    OUT out[16];  
  
    PARTS:  
  
    ///// Replace this comment with your code.  
    And(a=a[0], b=b[0], out=out[0]);  
    And(a=a[1], b=b[1], out=out[1]);  
    And(a=a[2], b=b[2], out=out[2]);  
    And(a=a[3], b=b[3], out=out[3]);  
    And(a=a[4], b=b[4], out=out[4]);  
    And(a=a[5], b=b[5], out=out[5]);  
    And(a=a[6], b=b[6], out=out[6]);  
    And(a=a[7], b=b[7], out=out[7]);  
    And(a=a[8], b=b[8], out=out[8]);  
    And(a=a[9], b=b[9], out=out[9]);  
    And(a=a[10], b=b[10], out=out[10]);  
    And(a=a[11], b=b[11], out=out[11]);  
    And(a=a[12], b=b[12], out=out[12]);  
    And(a=a[13], b=b[13], out=out[13]);  
    And(a=a[14], b=b[14], out=out[14]);  
    And(a=a[15], b=b[15], out=out[15]);  
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. At the top, the browser address bar shows the URL 'nand2tetris.github.io/web-ide/chip/'. The simulator's title bar indicates the project is 'NAND2Tetris' and the file being edited is 'And16.hdl'. The main workspace is divided into three panes. The left pane shows the HDL code for the And16 chip, which is a 16-bit AND gate. The code includes comments about its source and a testbench section. The middle pane shows a logic diagram of the chip, which is a simple AND gate. The right pane shows the chip's input and output pins. The input pins are labeled 'a' and 'b', and the output pin is labeled 'out'. The output pin value is shown as '0001000000110100'. The bottom status bar indicates 'Simulation successful: The output file is identical to the compare file'.

c. Mux4Way16:

```
CHIP Mux4Way16 {  
  
    IN a[16], b[16], c[16], d[16], sel[2];  
  
    OUT out[16];  
  
    PARTS:  
  
    Mux16(a=a , b=b , sel=sel[0] , out=ab );  
    Mux16(a=c , b=d , sel=sel[0] , out=cd );  
    Mux16(a=ab , b=cd , sel=sel[1] , out=out );  
  
}
```

The screenshot displays the Nand2Tetris Hardware Simulator interface. The top section shows the HDL code for the Mux4Way16 chip, which implements a 4-way 16-bit multiplexer using three Mux16 chips. The code includes comments and a PARTS section for chip instantiation.

The middle section shows the chip's configuration and simulation results. The input pins are set to a=0001001000110100, b=1001100001110110, c=1010101010101010, and d=0101010101010101. The select pins are set to sel=11. The output pin is out=0101010101010101. The internal pin ab is also shown.

The bottom section shows the test script, which sets the select pins and evaluates the output. The simulation is successful, and the output file is identical to the compare file.

Simulation successful: The output file is identical to the compare file

d. Mux8Way16:

```
CHIP Mux8Way16 {  
    IN a[16], b[16], c[16], d[16],  
        e[16], f[16], g[16], h[16],  
        sel[3];  
    OUT out[16];  
    PARTS:  
        Mux4Way16(a=a, b=b, c=c, d=d, sel=sel[0..1],  
out=abcd);  
        Mux4Way16(a=e, b=f, c=g, d=h, sel=sel[0..1],  
out=efgh);  
  
        Mux16(a=abcd, b=efgh, sel=sel[2], out=out);  
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top browser window shows the URL `nand2tetris.github.io/web-ide/chip/`. The simulator window is titled "NAND2Tetris / Hardware Simulator / Mux8Way16.hdl".

The main editor shows the HDL code for the `Mux8Way16` chip. The code includes comments and implements an 8-way 16-bit multiplexor using two `Mux4Way16` chips and a `Mux16` chip. The code is as follows:

```
1 // This file is part of www.nand2tetris.org  
2 // and the book "The Elements of Computing Systems"  
3 // by Nisan and Schocken, MIT Press.  
4 // File name: projects/1/Mux8Way16.hdl  
5 /**  
6  * 8-way 16-bit multiplexor:  
7  * out = a if sel = 000  
8  *       b if sel = 001  
9  *       c if sel = 010  
10 *       d if sel = 011  
11 *       e if sel = 100  
12 *       f if sel = 101  
13 *       g if sel = 110  
14 *       h if sel = 111  
15 */  
16 CHIP Mux8Way16 {
```

The right panel shows the "Chip Mux8Way16" configuration. It includes "Eval", "Reset", and "Clock: 0" buttons. Below these are the "Input pins" for inputs a through h, each with a 16-bit binary value and a "dec" button. The values are:

- a: 000100100001101000
- b: 001000011010000101
- c: 00110100001010110
- d: 0100010101100111
- e: 0101011001111000
- f: 0110011110001001
- g: 0111100010011010
- h: 1000100110101011

The bottom panel shows the "Test" section with a "Test Script" editor. The script contains the following code:

```
81  
82 set sel 6,  
83 eval,  
84 output;  
85  
86 set sel 7,  
87 eval,  
88 output;
```

Below the test script, a green status bar indicates: "Simulation successful: The output file is identical to the compare file".

4. LAB6:

a. DMux4Way:

```
CHIP DMux4Way {  
    IN in, sel[2];  
    OUT a, b, c, d;  
  
    PARTS:  
    DMux(in=in, sel=sel[1], a=left, b=right);  
    DMux(in=left, sel=sel[0], a=a, b=b);  
    DMux(in=right, sel=sel[0], a=c, b=d);  
}
```

The screenshot shows the NAND2Tetris Hardware Simulator interface. The top bar displays the project name "NAND2Tetris" and the file "DMux4Way.hdl". The main window is divided into three sections:

- Code Editor:** Displays the Verilog code for the DMux4Way chip. The code is as follows:

```
5 /**  
6  * 4-way demultiplexer.  
7  * [a, b, c, d] = [in, 0, 0, 0] if sel = 00  
8  *               [0, in, 0, 0] if sel = 01  
9  *               [0, 0, in, 0] if sel = 10  
10 *               [0, 0, 0, in] if sel = 11  
11 */  
12 CHIP DMux4Way {  
13     IN in, sel[2];  
14     OUT a, b, c, d;  
15  
16     PARTS:  
17     DMux(in=in, sel=sel[1], a=left, b=right);  
18     DMux(in=left, sel=sel[0], a=a, b=b);  
19     DMux(in=right, sel=sel[0], a=c, b=d);  
20 }
```
- Pin Configuration:** Shows the input and output pins for the chip. The input pins are "in" (value 1) and "sel" (values 1, 1). The output pins are "a" (0), "b" (0), "c" (0), and "d" (1). The internal pin "left" is also shown with a value of 0.
- Test Results:** Shows the test script and the output file. The test script is as follows:

```
34 output;  
35  
36 set sel %B10,  
37 eval,  
38 output;  
39  
40 set sel %B11,  
41 eval,  
42 output;
```

The output file is identical to the compare file, and the simulation is successful.

The bottom status bar indicates "Simulation successful: The output file is identical to the compare file".

b. DMux8Way:

```
CHIP DMux8Way {
```

```
    IN in, sel[3];
```

```
    OUT a, b, c, d, e, f, g, h;
```

```
    PARTS:
```

```
        DMux(in=in, sel=sel[2], a=left, b=right);
```

```
        DMux(in=left, sel=sel[1], a=ab, b=cd);
```

```
        DMux(in=right, sel=sel[1], a=ef, b=gh);
```

```
        DMux(in=ab, sel=sel[0], a=a, b=b);
```

```
        DMux(in=cd, sel=sel[0], a=c, b=d);
```

```
        DMux(in=ef, sel=sel[0], a=e, b=f);
```

```
        DMux(in=gh, sel=sel[0], a=g, b=h);
```

```
}
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top browser window shows the URL `nand2tetris.github.io/web-ide/chip/`. The simulator's title bar indicates the current project is `DMux8Way`. The main editor shows the HDL code for the `DMux8Way` chip, which is a multi-ported multiplexer. The code defines inputs `in` and `sel[3]`, and outputs `a` through `h`. It uses a `PARTS` block to instantiate several `DMux` components to implement the 8-way multiplexing logic. The right-hand panel shows the chip's configuration, with input pins `in` and `sel` set to 1, and output pins `a` through `f` set to 0. The bottom panel shows the test script, which sets the select lines to `%B110` and `%B111`, and evaluates the output. A green status bar at the bottom indicates that the simulation was successful.

```
15 */
16 CHIP DMux8Way {
17     IN in, sel[3];
18     OUT a, b, c, d, e, f, g, h;
19
20     PARTS:
21         DMux(in=in, sel=sel[2], a=left, b=right);
22
23         DMux(in=left, sel=sel[1], a=ab, b=cd);
24         DMux(in=right, sel=sel[1], a=ef, b=gh);
25
26         DMux(in=ab, sel=sel[0], a=a, b=b);
27         DMux(in=cd, sel=sel[0], a=c, b=d);
28         DMux(in=ef, sel=sel[0], a=e, b=f);
29         DMux(in=gh, sel=sel[0], a=g, b=h);
30 }
```

Test Script

```
67
68 set sel %B110,
69 eval,
70 output;
71
72 set sel %B111,
73 eval,
74 output;
```

Simulation successful: The output file is identical to the compare file

PROJECT 2 :

1. LAB1:

a. HalfAdder:

```
CHIP HalfAdder {  
    IN a, b;      // 1-bit inputs  
    OUT sum,      // Right bit of a + b  
        carry;   // Left bit of a + b
```

PARTS:

//// Replace this comment with your code.

Xor(a=a, b=b, out=sum);

And(a=a, b=b, out=carry);

}

The screenshot shows the Nand2Tetris Hardware Simulator interface. The top bar displays the URL 'nand2tetris.github.io/web-ide/chip/'. The main window is titled 'NAND2Tetris / Hardware Simulator / HalfAdder.hdl'. The left pane shows the HDL code for the HalfAdder chip, which includes comments and the implementation of the XOR and AND gates. The right pane shows the 'Chip HalfAdder' configuration with input pins 'a' and 'b' set to 1, and output pins 'sum' and 'carry' set to 0 and 1 respectively. The bottom pane shows the 'Test' results, which include a test script and a comparison of the output file with the compare file. A green message at the bottom states 'Simulation successful: The output file is identical to the compare file'.

```
1 // This file is part of www.nand2tetris.org  
2 // and the book "The Elements of Computing Systems"  
3 // by Nisan and Schocken, MIT Press.  
4 // File name: projects/2/HalfAdder.hdl  
5 /**  
6  * Computes the sum of two bits.  
7  */  
8 CHIP HalfAdder {  
9     IN a, b;      // 1-bit inputs  
10    OUT sum,      // Right bit of a + b  
11        carry;   // Left bit of a + b  
12  
13    PARTS:  
14    //// Replace this comment with your code.  
15    Xor(a=a, b=b, out=sum);  
16    And(a=a, b=b, out=carry);  
17 }
```

Test Script: 20 set a 1,
21 set b 0,
22 eval,
23 output;
24
25 set a 1,
26 set b 1,
27 eval,
28 output;

Simulation successful: The output file is identical to the compare file

b. FullAdder:

```
CHIP FullAdder {  
    IN a, b, c; // 1-bit inputs  
    OUT sum, // Right bit of a + b + c  
        carry; // Left bit of a + b + c
```

PARTS:

//// Replace this comment with your code.

```
HalfAdder(a=a, b=b, sum=s1, carry=c1);
```

```
HalfAdder(a=s1, b=c, sum=sum, carry=c2);
```

```
Or(a=c1, b=c2, out=carry);
```

```
}
```

The screenshot shows the NAND2Tetris Hardware Simulator interface. The top bar displays the project name "NAND2Tetris" and the file "FullAdder.hdl". The main editor shows the HDL code for the FullAdder chip, which includes comments and the implementation using HalfAdder and Or gates. The right panel shows the chip's pins: Input pins (a, b, c) and Output pins (sum, carry). The bottom panel shows the test script and the simulation results, indicating that the simulation was successful.

Test Script:

```
39 set b 1,  
40 set c 0,  
41 eval,  
42 output;  
43  
44 set c 1,  
45 eval,  
46 output;
```

Simulation successful: The output file is identical to the compare file

c. Add16:

```
CHIP Add16 {  
    IN a[16], b[16];  
    OUT out[16];  
    PARTS:  
    HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c0);  
    FullAdder(a=a[1], b=b[1], c=c0, sum=out[1], carry=c1);  
    FullAdder(a=a[2], b=b[2], c=c1, sum=out[2], carry=c2);  
    FullAdder(a=a[3], b=b[3], c=c2, sum=out[3], carry=c3);  
    FullAdder(a=a[4], b=b[4], c=c3, sum=out[4], carry=c4);  
    FullAdder(a=a[5], b=b[5], c=c4, sum=out[5], carry=c5);  
    FullAdder(a=a[6], b=b[6], c=c5, sum=out[6], carry=c6);  
    FullAdder(a=a[7], b=b[7], c=c6, sum=out[7], carry=c7);  
    FullAdder(a=a[8], b=b[8], c=c7, sum=out[8], carry=c8);  
    FullAdder(a=a[9], b=b[9], c=c8, sum=out[9], carry=c9);  
    FullAdder(a=a[10], b=b[10], c=c9, sum=out[10], carry=c10);  
    FullAdder(a=a[11], b=b[11], c=c10, sum=out[11], carry=c11);  
    FullAdder(a=a[12], b=b[12], c=c11, sum=out[12], carry=c12);  
    FullAdder(a=a[13], b=b[13], c=c12, sum=out[13], carry=c13);  
    FullAdder(a=a[14], b=b[14], c=c13, sum=out[14], carry=c14);  
    FullAdder(a=a[15], b=b[15], c=c14, sum=out[15], carry=c15);  
}
```

NAND2Tetris / Hardware Simulator / Add16.hdl

HDL ☐ --LBSst Project 2 Add16 Chip Add16 Eval Reset Clock: 0

```
1 // This file is part of www.nand2tetris.org  
2 // and the book "The Elements of Computing Systems"  
3 // by Nisan and Schocken, MIT Press.  
4 // File name: projects/2/Add16.hdl  
5 /**  
6  * 16-bit adder: Adds two 16-bit two's complement values.  
7  * The most significant carry bit is ignored.  
8  */  
9 CHIP Add16 {  
10     IN a[16], b[16];  
11     OUT out[16];  
12  
13     PARTS:  
14     /// Replace this comment with your code.  
15     HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c0);  
16
```

Input pins
a 0 0 0 0 1 0 0 1 0 0 0 0 1 1 0 1 0 0 dec
b 1 0 0 1 1 0 0 0 0 0 1 1 1 0 1 1 0 dec
Output pins
out 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 dec
Internal pins
c0 0
c1 0
c2 1
c3 0

Test ☐ ☐ ☐ Steps: 9 Outputs: 7 Slow Fast

Test Script Compare File Output File Diff Table

```
30 set a %0000000000000001,  
31 set b %0000011111111000,  
32 eval,  
33 output;  
34  
35 set a %00001001000110100,  
36 set b %0100110000110110,  
37 eval,  
38 output;
```

Simulation successful: The output file is identical to the compare file

```

d. Inc16:
    CHIP Inc16 {
        IN in[16];
        OUT out[16];

        PARTS:
        ///// Replace this comment with your code.
        Add16(a=in, b[0]=true, b[1..15]=false, out=out);
    }

```

The screenshot shows the NAND2Tetris Hardware Simulator interface. The top browser window displays the URL `nand2tetris.github.io/web-ide/chip/`. The simulator window is titled "NAND2Tetris / Hardware Simulator / Inc16.hdl".

The main editor shows the HDL code for the `Inc16` chip. The code includes comments about the source and a placeholder for the implementation. The implementation part is as follows:

```

CHIP Inc16 {
    IN in[16];
    OUT out[16];

    PARTS:
    ///// Replace this comment with your code.
    Add16(a=in, b[0]=true, b[1..15]=false, out=out);
}

```

On the right side, the "Chip Inc16" panel shows the input and output pins. The input pins are labeled "in" and the output pins are labeled "out". Both are currently set to 16-bit binary values: `1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1` for input and `1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0` for output. There are "dec" buttons next to each input/output field.

At the bottom, the "Test" panel shows a test script with the following steps:

```

10 output;
11
12
13
14
15
16
17
18 set in %B00000000000000101, // in = 5
19 eval,
20 output;
21
22 set in %B1111111111111011, // in = -5
23 eval,
24 output;

```

The simulation status bar at the bottom indicates: "Simulation successful: The output file is identical to the compare file".

2. LAB5:

a. ALU:

```
CHIP ALU {  
    IN  
        x[16], y[16], // 16-bit inputs  
        zx,nx,zy,ny,f,no;  
    OUT  
        out[16],zr,ng;  
    PARTS:  
        Mux16(a=x, b=false, sel=zx, out=x1);  
        Not16(in=x1, out=nx1);  
        Mux16(a=x1, b=nx1, sel=nx, out=x2);  
        Mux16(a=y, b=false, sel=zy, out=y1);  
        Not16(in=y1, out=ny1);  
        Mux16(a=y1, b=ny1, sel=ny, out=y2);  
        And16(a=x2, b=y2, out=fAnd);  
        Add16(a=x2, b=y2, out=fAdd);  
        Mux16(a=fAnd, b=fAdd, sel=f, out=fOut);  
        Not16(in=fOut, out=nfOut);  
        Mux16(a=fOut, b=nfOut, sel=no, out=out,  
out[0..7]=low8, out[8..15]=high8, out[15]=ng);  
        Or8Way(in=low8, out=orLow);  
        Or8Way(in=high8, out=orHigh);  
        Or(a=orLow, b=orHigh, out=notZero);  
        Not(in=notZero, out=zr);  
}
```

NAND2TETRIS - Google DocsNAND2TetrisAnd16 HDL CodeNew tabAsk Gemini

nand2tetris.github.io/web-ide/chip/

All Bookmarks

NAND2Tetris / Hardware Simulator / ALU.hdl

HDL--LBStProject 2ALU

Chip ALUEvalResetClock: 0

```
29 CHIP ALU {
55   Add16(a=x2, b=y2, out=fAdd);
56   Mux16(a=fAnd, b=fAdd, sel=f, out=f0Out);
57
58   Not16(in=f0Out, out=nf0Out);
59   Mux16(a=f0Out, b=nf0Out, sel=no, out=out, out[0..7]=low8, out[8..15]=high8, out[15]=ng);
60
61   Or8Way(in=low8, out=orLow);
62   Or8Way(in=high8, out=orHigh);
63   Or(a=orLow, b=orHigh, out=notZero);
64   Not(in=notZero, out=zr);
65
66   //// Replace this comment with your code.
67 }
68
```

Input pins

x	00000000000000100001	dec
y	000000000000000000011	dec
zx	0	
nx	1	
zy	0	
ny	1	
f	0	
no	1	

Output pins

Test

Steps: 41 Outputs: 37SlowFast

Test ScriptCompare FileOutput FileDiff Table

```
367 set zx 0,
368 set nx 1,
369 set zy 0,
370 set ny 1,
371 set f 0,
372 set no 1,
373 eval,
374 output;
```

Simulation successful: The output file is identical to the compare file